Project Documentation

1.Introduction

- **Project Title :** Product inventory management
- **Team ID: SWTID-2026-1360**
- **Team Leader**: P.Pooja pjayanthi521992@gmail.com
- **Team members:**

N. Krithika krithikanagarajan3@gmail.com

G.Nithya Dharshini nithyakathika23@gmail.com

T.Pradiksha pradiksha.thangaraju@gmail.com

2.Project overview:

The **Product Inventory Management System** is a backend-focused web application developed using **Node.js**, **Express.js**, and **MongoDB**, designed to provide a secure and scalable solution for managing product data through RESTful APIs. The system enables users to create accounts, log in securely, and access protected routes using **JWT-based authentication**, ensuring that only authorized users can perform operations. It supports complete **CRUD functionality**, allowing users to add new products, view their inventory, update product details such as price or quantity, and delete items when needed.

Key Goals:

- Provide secure JWT-based user authentication and authorization for protected API access.
- Enable efficient product inventory management through RESTful CRUD operations.
- Ensure user-specific data isolation so each user can manage only their own products.
- Implement scalable backend architecture using Node.js, Express.js, and MongoDB.

• Deliver fast and reliable API responses with proper validation and error handling.

Target Users:

• Small business owners managing product stock and inventory records.

• Freelancers or sellers tracking items or services in a structured way.

• Developers learning backend API development and authentication workflows.

• Students working on backend or full-stack development projects.

• Anyone needing a secure and simple backend system for managing product data.

PROJECT INSTRUCTION

- Create backend project folder
- Configure server using Express.js
- Setup MongoDB database connection
- Implement authentication APIs
- Implement product CRUD APIs
- Test APIs using Thunder Client in VS Code

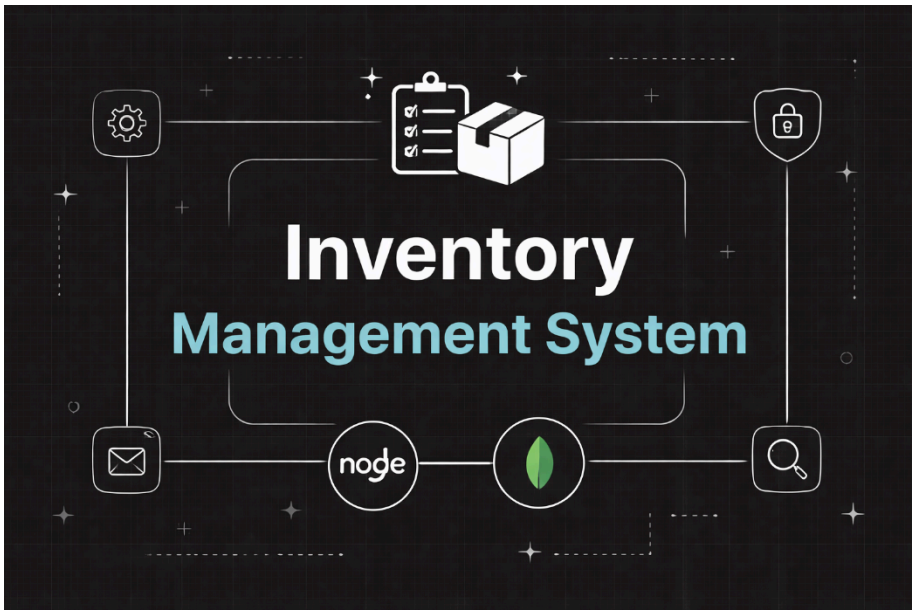
PRE-REQUISITES

Software	**Purpose**
Node.js	JavaScript runtime for backend
MongoDB	Database storage
npm	Package manager
VS Code	Development environment

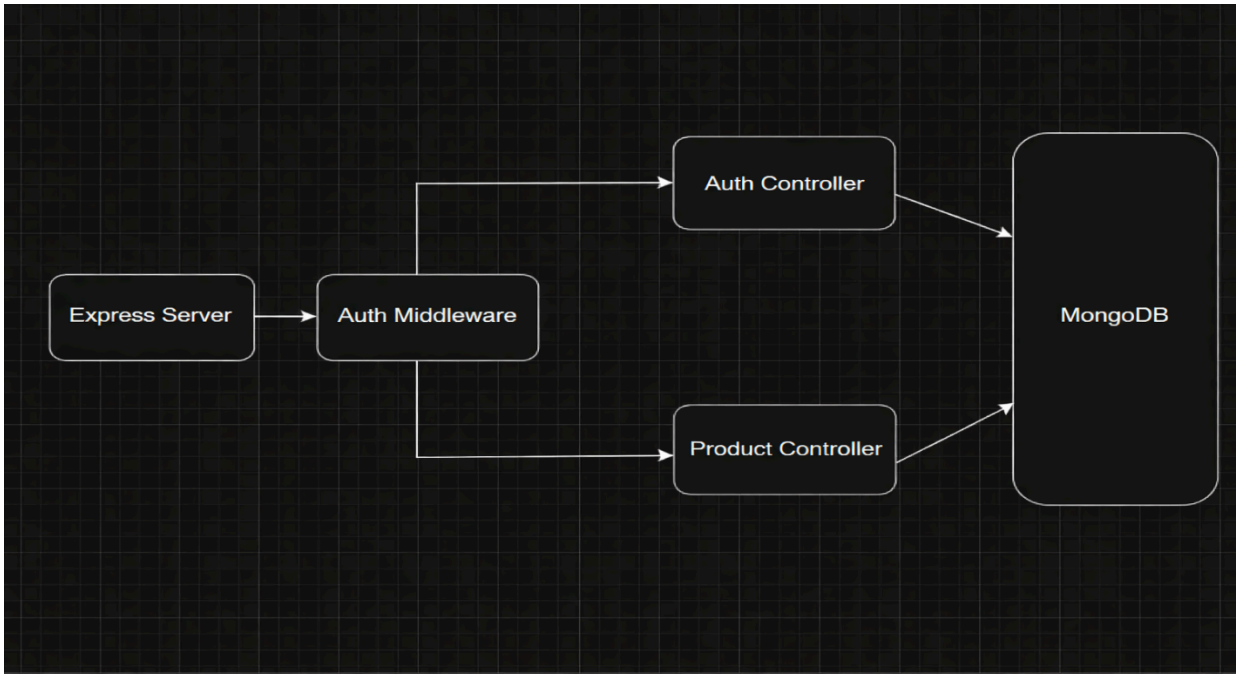
Knowledge Requirements

- JavaScript ES6 concepts
- Node.js and Express.js REST API development
- MongoDB and Mongoose ODM
- JWT authentication
- Middleware architecture

Project Thumbnail:



PROJECT ARCHITECTURE



High-level architecture diagram illustrating the three-tier backend structure.

Node.js + Express Backend

- RESTful API architecture
- JWT authentication middleware
- MVC pattern (Models, Controllers, Routes)
- Input validation and error handling

MongoDB Database

- Document-based NoSQL storage
- Mongoose ODM for schema modeling
- Two collections: Users and Contacts
- Indexed fields for optimized queries

TECHNICAL ARCHITECTURE

Backend Architecture (Node.js + Express)

- **Controllers**: Handle authentication and product logic
- **Models**: Define database schema

- **Routes**: API endpoints for authentication and products
- **Middleware**: JWT verification and error handling
- **Configuration**: Environment variables and database connection

Database Architecture (MongoDB)

Users Collection:

- _id
- name
- email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId (reference to user)
- name
- price
- quantity
- description
- timestamps

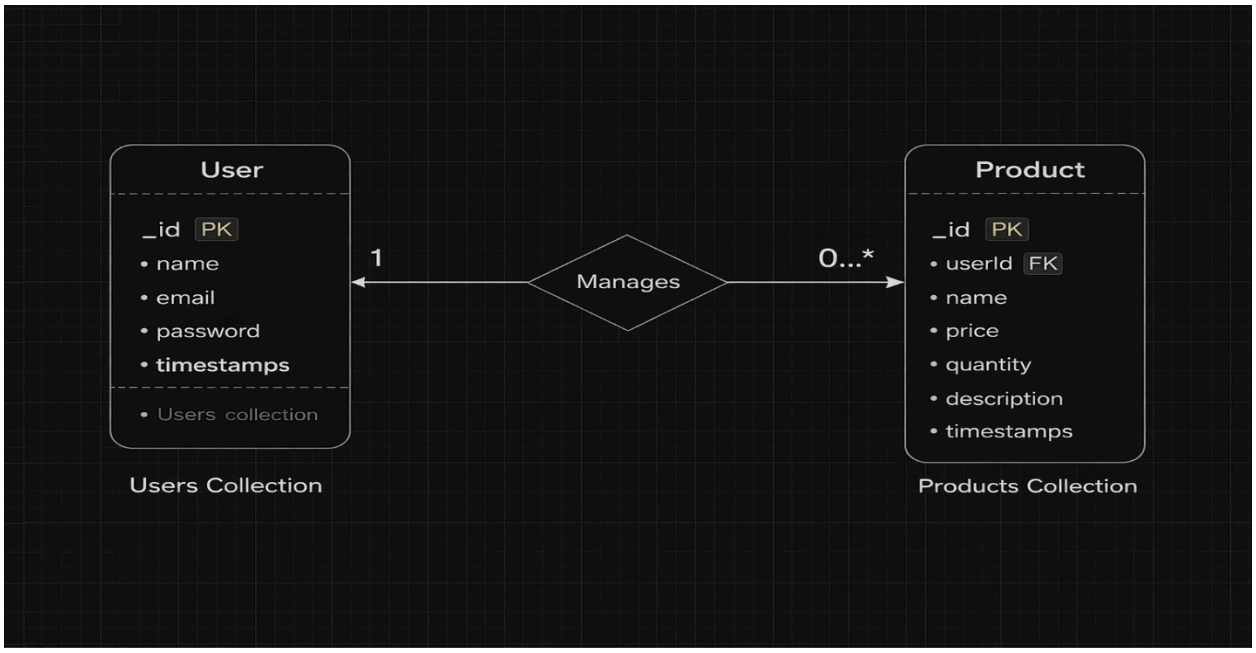
Relationship: One User → Many Products

Indexes:

- email (unique)
- userId
- product name

ER Diagram

ER diagram showing User → Product relationship:



Features:

Core Features

S.No	Feature	Description
1	User Authentication	JWT-based register and login system
2	Product CRUD	Add, view, update and delete products
3	Private Data Access	Users access only their own products
4	Product Search	Search products by name
5	Secure API	Middleware protected routes

Roles and Responsibility

User

Registration: Register using name, email and password

Login: Login using registered credentials

Add Product: Create product with name, price, quantity and description

Edit Product: Update existing product

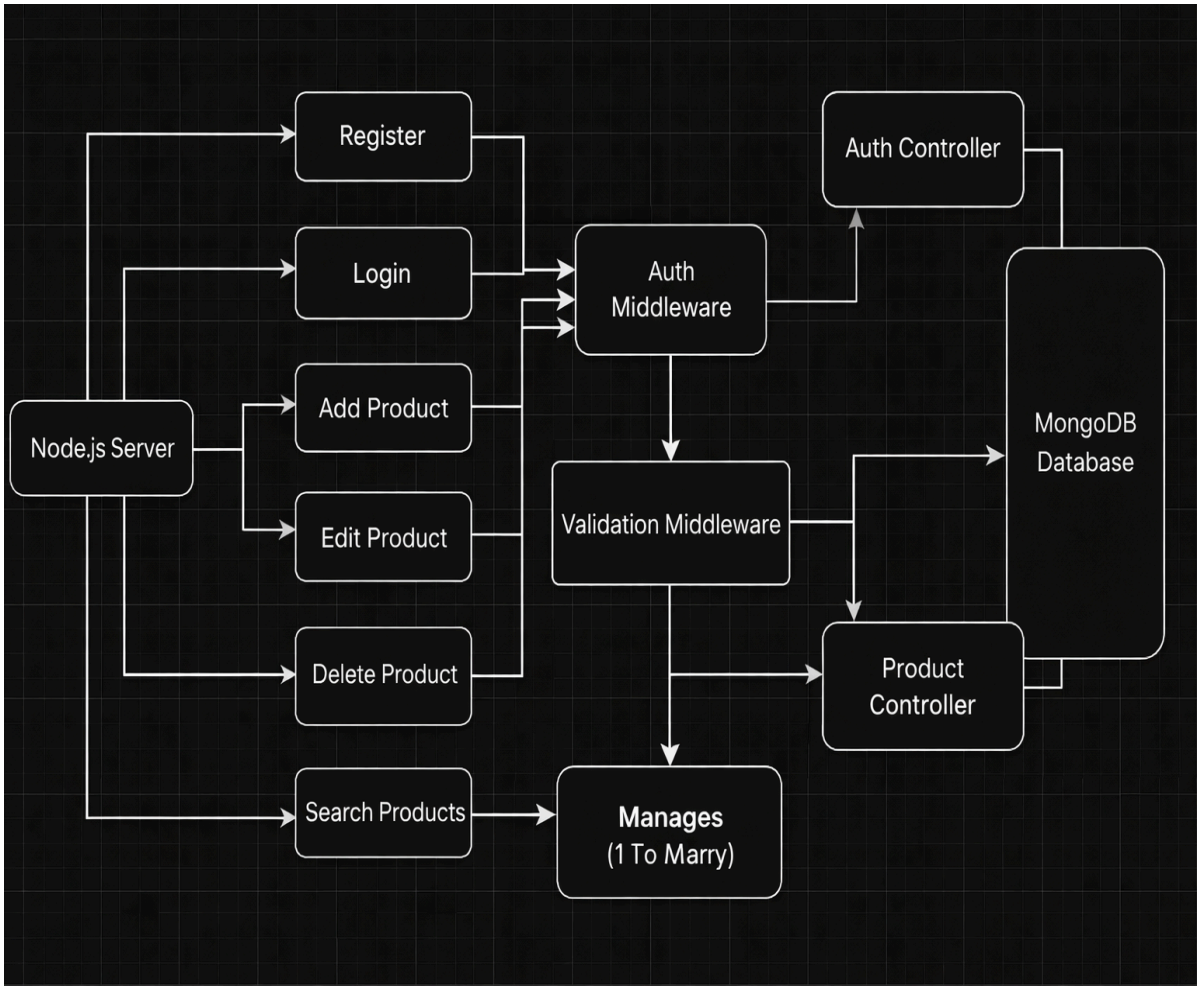
Delete Product: Remove product from inventory

Search Product: Search products using keyword

User Flow

The user sends a register/login request.

1. The backend generates a JWT token.
2. The user sends a token in the Authorization header.
3. Middleware verifies the token.
4. The controller performs CRUD operations.
5. MongoDB stores or retrieves product data.
6. API sends response.



Project Setup and Configuration

Folder Structure

```
server/
│
├── node_modules/
├── src/
│   ├── controllers/
│   ├── db/
│   ├──
```

```
middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
│
├── .env
├── .gitignore
├──
package-lock.json
└── package.json
```

Backend Development

Setup Express Server:

Inside src/server.js, create the basic Express server.

```
const express = require('express')
const app = express()
const port = 3000

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.listen(port, () => {
  console.log(`Example app listening on port ${port}`)
})
```

Product Controller

The **Product Controller** handles all product inventory operations and contains the business logic for managing products linked to individual users.

Responsibilities:

- Add new products to inventory
- Fetch products belonging to logged-in users

- Update product details like price, quantity, or description
- Delete products securely
- Perform product search functionality

Main Functionalities:

- **Create Product:** Adds a new product with a reference to the authenticated user.
- **Get Products:** Retrieves only the products created by the logged-in user.
- **Update Product:** Modifies product details while verifying ownership.
- **Delete Product:** Removes a product safely from the database.
- **Search Product:** Allows filtering products based on name.

Create Middleware

Inside: src/middlewares/

```
export const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authorized" });
  }

  try {
    const token = authHeader.split(" ")[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    req.user = decoded;
    next();
  } catch (error) {
    res.status(401).json({ message: "Token invalid" });
  }
};
```

Project Execution

Execution Steps

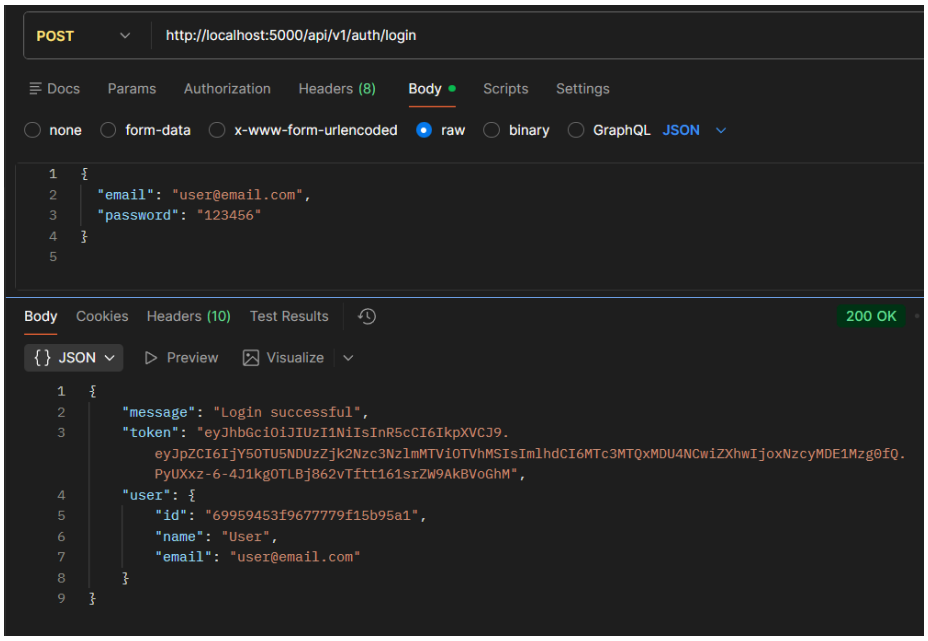
1. Open terminal in server folder.
2. Make sure you have a start script in the package.js file.
3. Run:

```
npm
start
```

Console should display:

```
Server running on
PORT
Database Connected
```

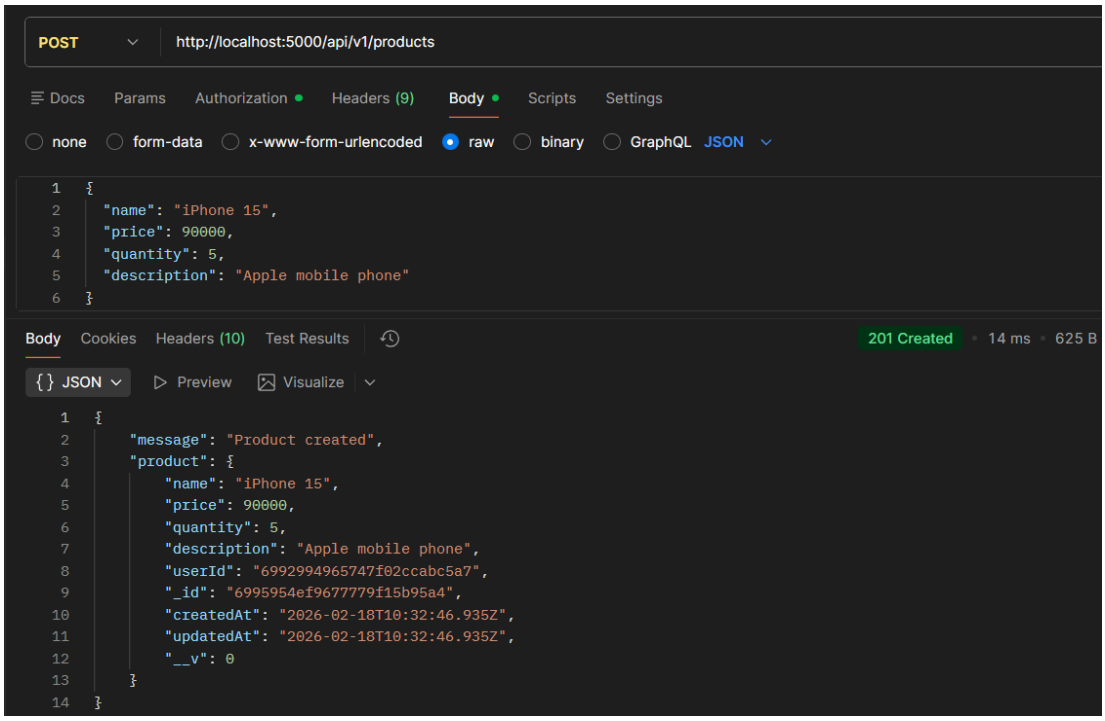
Login User



Login:

The login functionality verifies user credentials and generates a JWT token upon successful authentication. This token is used to access protected routes securely.

Create product



Create Product:

Allows an authenticated user to add a new product to their inventory by submitting product details. The product is saved with the user's ID to maintain ow

Get all Products

GET http://localhost:5000/api/v1/products

Docs Params Authorization Headers (7) Body Scripts Settings

Body Cookies Headers (10) Test Results 200 OK

{} JSON Preview Visualize


```
[
    {
        "_id": "6995954ef9677779f15b95a4",
        "name": "iPhone 15",
        "price": 90000,
        "quantity": 5,
        "description": "Apple mobile phone",
        "userId": "6992994965747f02ccabc5a7",
        "createdAt": "2026-02-18T10:32:46.935Z",
        "updatedAt": "2026-02-18T10:32:46.935Z",
        "__v": 0
    }
]
```


Get all Products:

Fetches all products that belong to the logged-in user. It ensures users can view only their own inventory data.

Update Product

PUT http://localhost:5000/api/v1/products/6995954ef9677779f15b95a4

Docs Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "name": "iPhone 19 Pro",
  "price": 95000,
  "quantity": 10,
  "description": "Updated phone"
}
```

Body Cookies Headers (10) Test Results 200 OK 37 ms

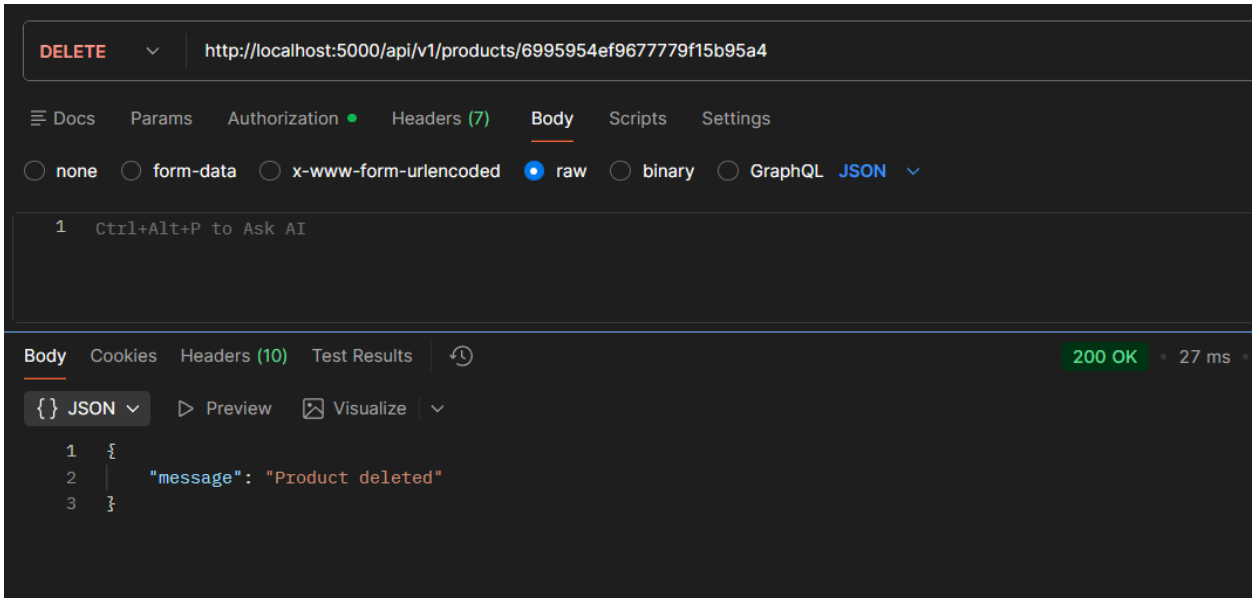
JSON Preview Visualize


```
{
    "message": "Product updated",
    "product": {
        "_id": "6995954ef9677779f15b95a4",
        "name": "iPhone 19 Pro",
        "price": 95000,
        "quantity": 10,
        "description": "Updated phone",
        "userId": "6992994965747f02ccabc5a7",
        "createdAt": "2026-02-18T10:32:46.935Z",
        "updatedAt": "2026-02-18T10:35:11.547Z",
        "__v": 0
    }
}
```


Update Product:

Enables users to modify existing product details such as name, price, or quantity. The system verifies ownership before allowing any updates.

Delete Product



Delete Product:

Allows users to permanently remove a product from their inventory. The operation is protected to ensure only the product owner can delete it.

[Demo Video]